Concept Paper — ReflexWork

งาน: Mahidol AI Community 2026 (MUIT × AWS) — Track: AI Adoption with Kiro **ด้านที่ส่ง:** การปรับปรุงกระบวนการทำงาน (Process Improvement) **ผู้พัฒนา:** ทีม 2 คน — Ratchapon (Ozone), Napatsakorn **Repository:** https://github.com/ozoneRatchapon/reflexwork

---

1. ชื่อผลงานและคำโปรย

ReflexWork — ชั้นตัดสินใจเร็วระดับไมโครวินาที ที่ลดต้นทุน AI ลง 80% พร้อม audit trail ตรวจสอบย้อนหลังได้

> ระบบจัดเส้นทางงาน (task routing) สำหรับหน่วยงานในมหาวิทยาลัย ที่คัดงาน deterministic ออกจาก LLM ก่อนตั้งแต่ต้นทาง จ่ายค่า AI เฉพาะงานที่ "ยาก" จริง และสร้างหลักฐานการตัดสินใจที่ตรวจสอบย้อนหลังได้ทุกครั้ง — สร้างด้วย Kiro + AI (Amazon Bedrock ผ่าน MCP สำหรับ Kiro IDE / Typhoon ในเครื่องสำหรับเว็บ)

---

2. ปัญหา (Problem)

หน่วยงานในมหาวิทยาลัย (งานทะเบียน งานบุคคล งานการเงิน งานบริการนักศึกษา) กำลังเริ่มนำ LLM/Generative AI มาช่วยงานเอกสารและคำร้อง แต่พบปัญหา 3 ข้อที่ เป็นอุปสรรคต่อการนำไปใช้จริงในวงกว้าง:

1. **ต้นทุนบานปลายและคาดเดายาก** — ทุกคำร้อง/ทุก task ถูกส่งเข้า LLM หมด ทั้งที่ส่วนใหญ่เป็นงานซ้ำ ๆ ที่ตัดสินใจด้วยกฎได้ (เช่น คัดประเภทคำร้อง, ตรวจความครบของเอกสาร, จัดลำดับความเร่งด่วน). จ่าย token ให้งานที่ไม่จำเป็น ต้องใช้ AI คือการเผาเงินงบประมาณ
2. **ความช้าและคอขวด** — การเรียก LLM ผ่านเครือข่ายใช้เวลา 350–800 ms ต่อครั้ง เมื่อปริมาณงานสูง (ช่วงเปิดเทอม/ลงทะเบียน) ระบบกลายเป็นคอขวด
3. **ตรวจสอบย้อนหลังไม่ได้ (ขาด accountability)** — เมื่อ AI ตัดสินใจแทนคน หน่วยงานภาครัฐ/มหาวิทยาลัยต้องตอบได้ว่า "ตัดสินใจแบบนี้เพราะอะไร ด้วยข้อมูลใด" เพื่อให้สอดคล้องกับ PDPA และหลักธรรมาภิบาลข้อมูล แต่ LLM แบบ black-box ให้คำตอบที่ reproduce ไม่ได้และไม่มีร่องรอยพิสูจน์

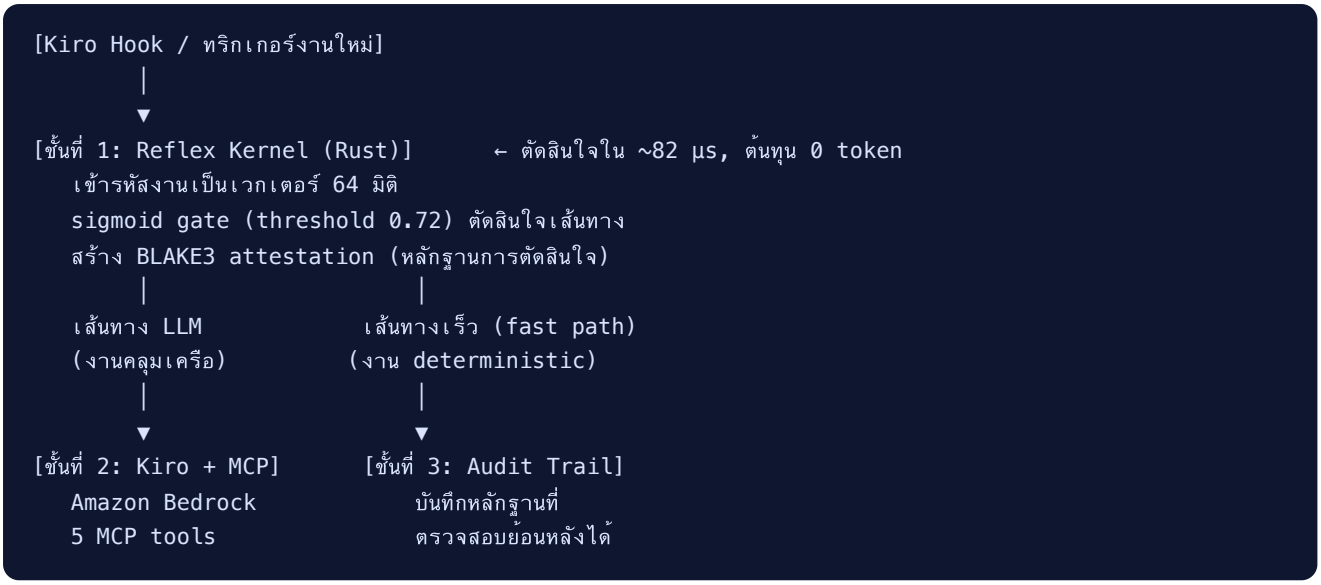
แก่นของปัญหา: รูปแบบ "เรียก LLM ทุกครั้ง" (always-call-the-LLM) แพง ช้า และตรวจสอบไม่ได้ — ไม่เหมาะกับการ scale ในองค์กรที่มีงบจำกัดและต้องการ accountability

---

3. แนวคิดและวัตถุประสงค์ของ Solution

ข้อสังเกตหลัก: งาน workflow ในองค์กรส่วนใหญ่ (> 80%) เป็นงาน deterministic ที่ตัดสินใจได้โดยไม่ต้องเรียก LLM เลย ควรมี "ชั้นปฏิกิริยาเร็ว" (reflex layer) มาคัดกรองก่อน และเรียก LLM เฉพาะงานที่คลุมเครือจริง ๆ

ReflexWork เป็นสถาปัตยกรรม 3 ชั้น:



วัตถุประสงค์:

1. ลดต้นทุน LLM โดยคัดงาน deterministic (> 80%) ออกก่อน → จ่ายเฉพาะงานที่ยากจริง
2. ตอบสนองงานทั่วไปในระดับไมโครวินาที (fast path) ไม่ต้องรอเครือข่าย
3. สร้าง **audit trail ที่ reproduce ได้** (BLAKE3 hash chain) ทุกการตัดสินใจ — เปิดกล่องดำของ AI ให้ตรวจสอบได้ สอดคล้อง PDPA

หัวใจทางเทคนิค (จากโค้ดจริงใน repo):

- `FeatureEncoder` — แปลง `TaskSpec` เป็นเวกเตอร์ `f32` 64 มิติ (ประเภทงาน, ความซับซ้อน, งบ token คงเหลือ, ความเร่งด่วน, keyword tags)
- `PolicyRouter` — sigmoid gate `g = σ(wᵀx + b)` ; ถ้า `g < 0.72` → fast path, ไม่งั้น → เรียก LLM. ขยายเป็น softmax 4 เส้นทางได้
- `StateAttestor` — BLAKE3 hash chain: `BLAKE3(input_hash ‖ state_hash ‖ output_hash)` = หลักฐานการตัดสินใจที่ใครก็ตรวจซ้ำได้
- การเรียนรู้ออนไลน์แบบ EMA (`ŵ = α·w + (1−α)·r·x` , α=0.97) — ปรับตัวตามผลงานจริง โดยไม่ต้อง retrain และไม่เรียก LLM

> แนวทางนี้สอดคล้องกับทิศทางงานวิจัยปัจจุบัน (เช่น gist.rs/Reflex) ที่เสนอ "decision engine ที่รันบนเครื่อง ฟรี ระดับ µs และ *abstain* เมื่อไม่มั่นใจ แทนที่จะเดา" ReflexWork นำหลักเดียวกันมาใช้ในบริบทกระบวนการทำงานของมหาวิทยาลัย

---

4. กลุ่มผู้ใช้ / ผู้ได้รับประโยชน์

กลุ่ม	ได้ประโยชน์อย่างไร
เจ้าหน้าที่หน่วยงาน (ทะเบียน/บุคคล/การเงิน)	งานคัดกรอง/จัดลำดับคำร้องที่ซ้ำ ๆ ถูกจัดการอัตโนมัติในระดับ µs เหลือเวลาไปทำงานที่ต้องใช้วิจารณญาณ
ผู้บริหาร / หัวหน้างาน	เห็น audit trail ทุกการตัดสินใจของ AI ตอบคำถามตรวจสอบได้ ควบคุมงบ token ได้จริง
หน่วยงาน IT (เช่น MUIT)	ลดภาระ cost ของ LLM, ติดตั้งเป็น service กลางที่หน่วยงานอื่นเรียกใช้ซ้ำได้
นักศึกษา/ผู้ยื่นคำร้อง	คำร้องทั่วไปได้รับการตอบสนองเร็วขึ้น ไม่ติดคอขวดช่วง peak
ผู้ตรวจสอบ / งาน Compliance (PDPA)	ตรวจย้อนหลังได้ว่า AI ตัดสินใจด้วยข้อมูลใด reproduce ผลเดิมได้

---

5. การประยุกต์ใช้ AI และ Kiro

ReflexWork ใช้ **Kiro เป็นเครื่องมือหลักในการพัฒนาและเป็นสมองของชั้นที่ 2**:

5.1 Kiro Hooks — ขับเคลื่อน workflow อัตโนมัติ

- `.kiro/hooks/on-task-created.json` — ทริกเกอร์เมื่อมีไฟล์ task ใหม่ → ส่งเข้า Reflex Kernel เพื่อ route
- `.kiro/hooks/on-proof-ready.json` — ทริกเกอร์เมื่อ attestation พร้อม → บันทึก audit

5.2 Kiro MCP Server — 5 เครื่องมือเรียกได้จาก Kiro IDE โดยตรง

ลงทะเบียนใน `.kiro/settings.json` (เซิร์ฟเวอร์ชื่อ `reflexwork`):

MCP Tool	ทำอะไร	AWS Service
`analyze_task`	วิเคราะห์ความซับซ้อนของงาน	Amazon Bedrock (Claude)
`synthesize_document`	สรุป/ร่างเอกสาร	Amazon Bedrock (Claude)
`query_knowledge_base`	ค้นหา context จากฐานความรู้	Bedrock Knowledge Bases
`draft_workflow`	ออกแบบขั้นตอน workflow	Amazon Bedrock (Claude)
`attest_and_submit`	สร้าง + บันทึกหลักฐานการตัดสินใจ	Bedrock + ชั้น audit

5.3 บทบาทของ AI (เรียกเฉพาะเส้นทาง LLM)

AI (LLM) ถูกเรียกเฉพาะงานที่ Reflex Kernel ตัดสินว่า "คลุมเครือ" (gate ≥ 0.72) เท่านั้น เช่น การเรียบเรียงคำแนะนำจากคู่มือ (grounded hybrid) — เป็นการใช้ AI อย่างคุ้มค่า ไม่ใช่เรียกทุกครั้ง

เส้นทาง LLM ที่รันจริงตอนนี้ (ผ่าน `_shared/team-adapter.mjs`): รองรับหลาย provider — **Typhoon (Thai-tuned) รันในเครื่องผ่าน Ollama** (0 token, ข้อมูลไม่ ออกจากเครื่อง — PDPA), Gemini (ผ่าน proxy), และสล็อตสำหรับ Amazon Bedrock. ทุก เส้นทางใช้ prompt แบบ **grounded เท่านั้น** ("ห้ามแต่งนอกคู่มือ") — AI เรียบเรียง ไม่ประดิษฐ์ข้อเท็จจริง

5.4 การสร้างด้วย Kiro (spec-driven) + ความสามารถที่ต่อยอดได้

- **Kiro spec-driven workflow** — ทุกฟีเจอร์ใหญ่มี `.kiro/specs/<feature>/{requirements,design,tasks}.md` (learning-loop, voice, distributable-kernel) + `.kiro/steering/{product,tech,structure}.md` เป็น always-on

context (ดู `docs/kiro-ai-sdlc.md`)

- **Self-improving KB** — ยิ่งใช้ยิ่งฉลาด: คำถามที่ตอบไม่ได้ → backlog (mask PII), 👎 ลด confidence ของ entry, auto-draft คำตอบให้ staff กด approve — ไม่ retrain
- **แจกจ่ายได้ (distributable)** — `@reflexwork/kernel` (npm) + `reflexwork init`: หน่วยงานอื่นติดตั้ง kernel ไปทำแชตบอตบน KB ตัวเองได้
- **Federated corpus** — หลายหน่วยงานช่วยกันโต corpus กลางผ่าน **signal ที่ mask แล้ว** (gap buckets/counts — ไม่มี raw text), draft ต้อง human approve, release เซ็นชื่อ + **anchor บน Solana** ให้ตรวจสอบข้ามองค์กรได้ (program deploy จริงบน devnet แล้ว)

---

6. คุณค่าและโอกาสในการนำไปใช้งานจริง (Feasibility)

6.1 ผลลัพธ์ที่วัดได้แล้ว (จากโค้ดจริง + benchmark)

Metric	ค่าที่วัดได้	หลักฐาน
`policy_route` (ตัดสินใจเส้นทาง)	**~12.2 ns**	`cargo bench --bench policy_routing` (วัดสด)
`feature_encode` (เข้ารหัสเวกเตอร์)	**~63.3 ns**	เดียวกัน (วัดสด)
Fast-path latency (end-to-end kernel)	**< 100 µs**	encode + route + BLAKE3 attest
ต้นทุน token เส้นทางเร็ว	**0 tokens**	ออกแบบไว้ให้ > 80% ของงานไม่แตะ LLM
หลักฐานการตัดสินใจ	**BLAKE3 reproducible**	`crates/reflex-kernel/src/attestation.rs`
KB retrieval (neurosymbolic TF-IDF)	**recall@1 = 82% · recall@3 = 96%**	eval 50 คำถามจริง `tests/kb_search.test.mjs` (วัดสด)
off-corpus abstain (ไม่เดา)	**2/2 ถูก · false-positive ≤ 6/50**	eval เดียวกัน

> **การค้นคู่มือเป็น neurosymbolic จริง (วัดก่อนเลือก):** เทียบ 3 วิธีบนคอร์ปัสจริง — SimHash (recall@1 1/11, ตัดทิ้ง) · lexical ล้วน (5/11) · **TF-IDF cosine (6/11@1, 9/11@3 → ผู้ชนะ)** จึงเลือก TF-IDF เป็นหลัก + ใช้ lexical overlap เป็นสัญญาณ symbolic เสริม. การรวม สองสัญญาณนี้ (overlap-aware gate) แก้เคสจริงที่ผู้ใช้เจอ — "Turnitin เกิน 25% ทำอย่างไร" ที่ เดิมได้คะแนนไม่มั่นใจ (cosine เจือจางเพราะ keyword ยาว) ให้กลับมาตอบมั่นใจ (s=6).

> ตัวเลข ns ข้างบนวัดสดด้วย Criterion บนเครื่องพัฒนา (Apple Silicon) ค่าจริงขึ้นกับฮาร์ดแวร์ จุดสำคัญคือ **การตัดสินใจเส้นทางอยู่ในระดับนาโน-ไมโครวินาที** เทียบกับการเรียก LLM ที่ใช้ 350–800 ms

6.2 ทำไมถึงนำไปใช้จริงในมหาวิทยาลัยได้

- **เบาและพกพาได้** — Reflex Kernel เป็น Rust คอมไพล์เป็น native หรือ WebAssembly (เป้า memory ≤ 2 MB) ติดตั้งเป็น sidecar/service กลางได้โดยไม่ผูกกับ cloud ตัวใด
- **ลดต้นทุนจริง** — ยิ่งปริมาณงานสูง การคัด 80% ออกจาก LLM ยิ่งประหยัดเป็นสัดส่วน
- **สอดคล้อง PDPA/ธรรมาภิบาล** — audit trail ที่ reproduce ได้ ตอบโจทย์การตรวจสอบ
- **ต่อยอดจากของที่มี** — ใช้ Amazon Bedrock ที่ MUIT × AWS มีอยู่แล้ว ไม่ต้องตั้ง stack ใหม่

6.3 ขอบเขต Prototype ปัจจุบัน และงานต่อยอด (ตรงไปตรงมา)

- **ทำงานได้จริงแล้ว (รันสด/ตรวจได้):**
  - Reflex Kernel (routing + BLAKE3 + EMA) + benchmark
  - แชตบอต + แดชบอร์ด **live** ที่ `reflexwork.pages.dev` (WASM kernel ในเบราว์เซอร์)
  - **self-improving KB (learning loop)** — gap log (PII-masked) + entry confidence (EMA) + auto-draft → human approve
  - **EMA-bandit จาก 👍/👎** — สัดส่วน FastPath ปรับตัวเองจาก feedback ผู้ใช้แบบ exponential moving average (deterministic, 0 token)
  - **UX แยกชั้น user/หลักฐาน** — หน้าแชตแบ่งสองคอลัมน์: ซ้าย = สิ่งที่ผู้ใช้เห็น (คำตอบสะอาด), ขวา = หลักฐานที่ตรวจสอบได้ (เส้นทาง · คะแนน · หัวข้อ KB · BLAKE3 proof) ต่อแต่ละข้อความ — สื่อจุดขาย auditable โดยไม่รกหน้าผู้ใช้
  - **Feature Coverage Radar** — สรุป coverage ต่อมิติของเราเอง (FastPath 95% · Audit 95% · KB 90% · Abstain 90% …) ในสไลด์/one-pager
  - **@reflexwork/kernel** — แพ็กเป็น npm lib ติดตั้งไปใช้ได้จริง (smoke test ผ่าน) + `reflexwork init` scaffolder
  - **federated corpus** — หลาย install ช่วยกันโต corpus กลางผ่าน signal ที่ mask แล้ว (ไม่มี raw text ออกจากเครื่อง); two-install demo รันได้จริง
  - **Solana program `workflow-escrow` deploy จริงบน devnet** (`6Dzv8va1Se8…tYE5`) — anchor hash ของ corpus release ให้ตรวจสอบข้ามองค์กรได้
- **เป็น stub/future work:** เชื่อมฐานข้อมูลคำร้องจริงของหน่วยงาน, UI review สำหรับ approve corpus, deploy anchor-webhook signer สู่ production
- **การอ่านออกเสียง (voice):** ถอดออกจาก prototype ชั่วคราว — ยังไม่พร้อมใช้งานจริง จะกลับมาทำเมื่อเสถียร

6.4 งานที่เกี่ยวข้องและข้อจำกัดที่ยอมรับตรง ๆ (related work)

ReflexWork ไม่ได้คิด "บันไดตัดสินใจ" ขึ้นเอง แต่ต่อยอดจากสาย **Reflex / Instinct / Rethink ของ katopz (gist.rs, MIT)** — decision engine แบบไม่ใช้โมเดล ที่ตอบระดับ ไมโครวินาที, **abstain เมื่อไม่มั่นใจ**, และมี receipt ตรวจย้อนได้ทุกคำตอบ — และได้บทเรียนเพิ่มจาก field notes ภาษาไทยของ **Megawiz/Asgard**. รายละเอียดการเทียบและ เครดิตอยู่ใน `docs/related-work.md`. สิ่งที่เรายืมมาโดยตรง: หลักบันได (จ่ายเฉพาะที่ ชั้นล่างตอบไม่ได้), abstain เป็นคำตอบ, receipt/proof ต่อคำตอบ, การแก้ tokenizer ไทย ด้วย n-gram ระดับตัวอักษร, และ structural guard ("system prompt คือมารยาท ไม่ใช่ กำแพง" → เราใส่ guardrail ตัดข้อเท็จจริงที่ไม่อยู่ใน KB).

ข้อจำกัดที่ยอมรับตรง ๆ (ตามวัฒนธรรม "พิสูจน์ก่อน อวยทีหลัง"):

- **threshold (gate θ=0.72, KB-score cutoff) ยังไม่ได้ fit จาก labeled data** — เป็นค่า default ที่มีเหตุผล แต่ Reflex skill เตือนไว้ว่า "default thresholds do not transfer" การ fit จากชุดที่คนไทย label คือ next step อันดับแรก
- **ยังไม่มีชุดประเมินภาษาไทยที่คน label** — ตัวเลขที่อ้างเป็นเชิงสถาปัตยกรรม ยังไม่ใช่ benchmark; ถ้าจะอ้าง accuracy จะกำกับ n + 95% CI เสมอ
- **learning loop ของเราออกแบบให้ human approve ทุก draft โดยตั้งใจ** — เพราะ Asgard พิสูจน์ว่า "ให้ระบบเรียนจากคำตอบตัวเอง (naive) = ทางลัดสู่ความผิดที่มองไม่ เห็น"; เราจึงเปลี่ยน corpus (ไม่ใช่ weights) และให้คนเป็นผู้อนุมัติเสมอ

ขอบคุณ katopz ที่เปิดงานนี้ และทีม Megawiz/Asgard สำหรับ field notes ที่ทำให้บันได ของเราดีขึ้น.

---

7. การสาธิต (Demo)

7.1 ทางเข้าใช้งาน (User Journey) — ใครใช้ระบบนี้ยังไง

ReflexWork เปิดให้ใช้งานได้ 3 ทางตามกลุ่มผู้ใช้:

ผู้ใช้	ใช้ยังไง	ช่องทาง (รันได้วันนี้)
🎓 **นักศึกษา / ผู้ยื่นคำร้อง**	ยื่นคำร้องผ่านพอร์ทัล → ระบบคัดกรอง/ตอบรับทันที ระดับ µs	REST API `POST /route`
👨‍💼 **เจ้าหน้าที่หน่วยงาน**	วางไฟล์งานลงโฟลเดอร์ให้ route อัตโนมัติ หรือเรียกเครื่องมือจาก Kiro IDE	Kiro Hook · Kiro IDE + MCP · CLI `reflex-agent route`
👔 **ผู้บริหาร / ผู้ตรวจสอบ**	ดูสถิติสรุป + ตรวจ audit trail (re-run BLAKE3 ได้ผลเดิมเสมอ)	REST API `GET /status` · BLAKE3 audit

เส้นทางการใช้งานจริง (end-to-end):

```
พอร์ทัล / ไฟล์งาน
      │
      ▼
REST API (reflex-serve)  หรือ  Kiro Hook        ← ทางเข้า
      │
      ▼
Reflex Kernel  ──fast path (µs)──►  ตอบกลับทันที (คัดประเภท/ตรวจครบ/จัดลำดับ) + BLAKE3 proof
      │
      └──LLM path──►  Kiro MCP → Amazon Bedrock  (สรุป/ร่างเอกสารที่ซับซ้อน)
      │
      ▼
Audit Trail (BLAKE3 reproducible)  ──►  Dashboard ผู้บริหาร / ผู้ตรวจสอบ
```

7.2 รันสาธิต

ก. ลองในเบราว์เซอร์ทันที — เปิด URL เดียว ไม่ต้องติดตั้งอะไร (เหมาะกับกรรมการ/ผู้ใช้ทั่วไป):

🌐 **ลองสดได้ที่ https://reflexwork.pages.dev** (deploy บน Cloudflare Pages)

- **ลองใน browser** — reflexwork.pages.dev/try.html : พิมพ์งาน → เห็น route + BLAKE3 proof สด
- **แชตบอต** — reflexwork.pages.dev/chatbot.html : ถาม-ตอบลงทะเบียน/วิทยานิพนธ์
- **ยื่นคำร้อง** — reflexwork.pages.dev/student-desk.html : ฟอร์มคำร้อง + มุมมองเจ้าหน้าที่
- **แดชบอร์ด** — reflexwork.pages.dev/dashboard.html : สถิติ fast/LLM + audit trail

Reflex Kernel ตัวจริง **compile เป็น WebAssembly** รันในเบราว์เซอร์ 100% ไม่มี backend ไม่ส่งข้อมูลออกไปไหน — algorithm เดียวกับฝั่ง server เป๊ะ. ทุกหน้าใช้ **kernel ตัวเดียวกัน** ต่างแค่มุมมองสำหรับผู้ใช้คนละกลุ่ม เชื่อมถึงกันผ่านแถบเมนูด้านบน

ข. REST API (สำหรับต่อระบบจริงของหน่วยงาน):

```
cargo build --release --bin reflex-serve
PORT=8787 ./target/release/reflex-serve      # bind 127.0.0.1 เท่านั้น

curl -X POST localhost:8787/route \
  -H 'Content-Type: application/json' \
  -d '{"category":"ComplianceCheck","input":"ตรวจคำร้องว่าเอกสารครบไหม"}'
# → { "route":"FastPath", "kernel_us":3.46, "blake3_proof":"...", "token_cost":"0 tokens ..." }
```

dashboard ผู้บริหาร (reflexwork.pages.dev/dashboard.html) โชว์สัดส่วน fast/LLM + audit trail แบบ real-time เมื่อต่อกับ REST API ที่รันอยู่

ค. CLI (interactive):

```
reflex-agent route --category ComplianceCheck --input "..."
```

ง. end-to-end พร้อม on-chain variant (optional):

```
./demo.sh           # หรือ ./demo.sh --bench
```

Demo แสดง: การตัดสินใจของ kernel แยก 3 ทาง (FastPath / McpAnalyze / McpSynthesize) พร้อม latency, BLAKE3 composite hash, และการ route งานผ่าน Kiro MCP

---

8. สรุป

ReflexWork เปลี่ยนรูปแบบการใช้ AI ในองค์กรจาก "เรียก LLM ทุกครั้ง" (แพง-ช้า-ตรวจสอบไม่ได้) มาเป็น "คัดด้วย reflex ก่อน เรียก AI เฉพาะที่จำเป็น และพิสูจน์ได้ทุกการตัดสินใจ" — ตอบโจทย์การปรับปรุงกระบวนการทำงานของมหาวิทยาลัยทั้งด้านต้นทุน ความเร็ว และ accountability โดยใช้ Kiro เป็นแกนหลักในการพัฒนา และเรียก AI อย่างคุ้มค่า (Amazon Bedrock ผ่าน MCP สำหรับ Kiro IDE, หรือ Typhoon ในเครื่องสำหรับเว็บ — ข้อมูลไม่ออกจากเครื่อง)